Web Systems Development

React + Laravel: wiring the front end to the back end

Session 18 · Fri 10/30 · taught from project/pantry-example-final

RPI

TODAY · TWO PARTS

Where we are, and where the example goes next

PART 1 · WHAT WE HAVE DONE SO FAR

1. **The finished product**
   What we are building toward, running.
2. **The contract first**
   Why the midterm made this easy.
3. **Homework 7: start api/**
   The table, the model, the seed data, the read routes.
4. **Next steps for your team**

PART 2 · THE FUTURE (WEEKS 11–13)

5. **fetch() and the proxy**
   api.js changes; why there is no CORS to fight.
6. **Logging in**
   Sessions, cookies and CSRF with Sanctum.
7. **The Laravel side**
   Policy, validation, controller, resource.
8. **Errors and tests**
   401, 403, 404, 422 on the page; three suites.

Part 1 · What we have done so far

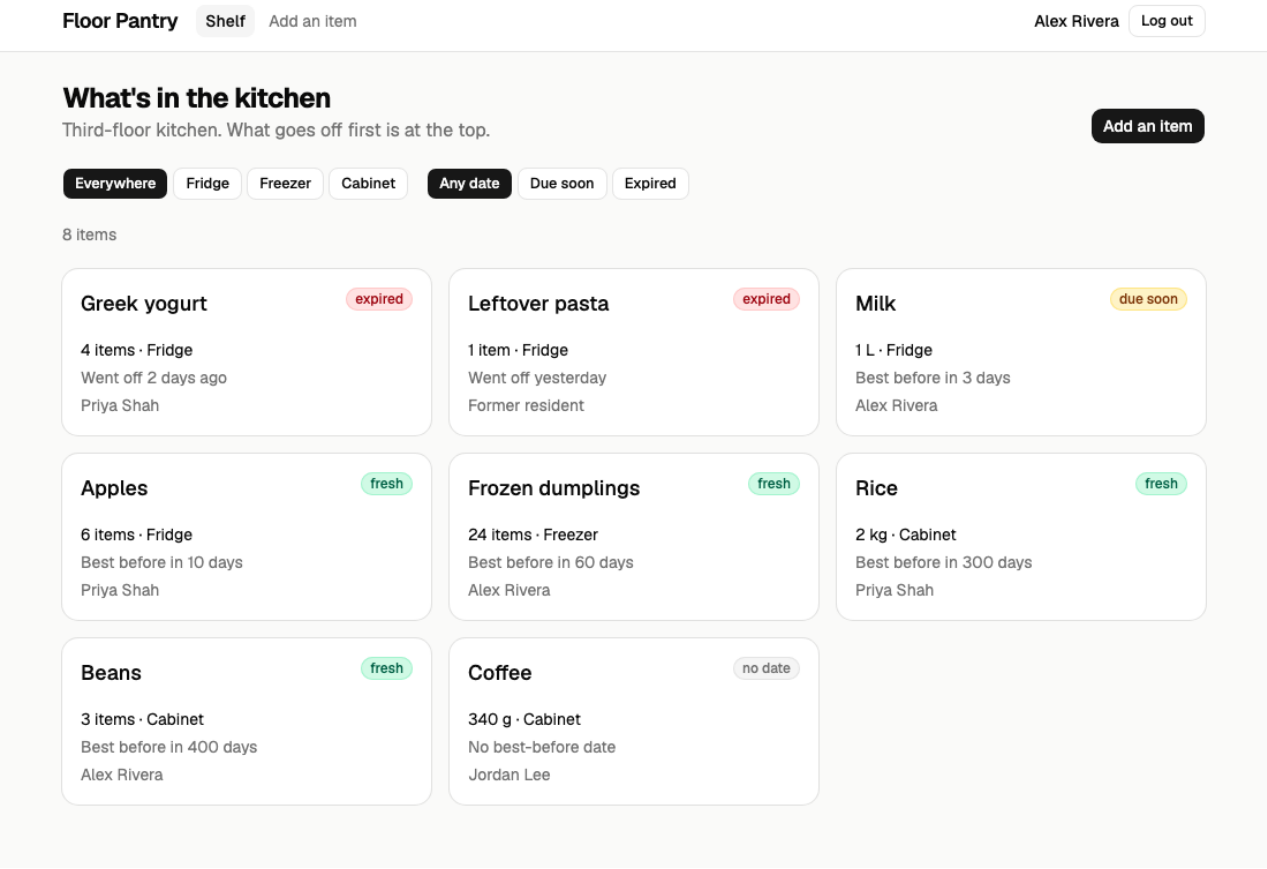
Docker, tests, React, Laravel routes and Eloquent, and the midterm contract. The pieces of the final example you already know, and what Homework 7 adds this week.

The finished product

Floor Pantry, the project example, with a real back end. Everything today is a piece of this.

SHOW THE END FIRST

What the final example does



SAME PAGES AS THE MIDTERM

The shelf, an item, the form. Not one page was rewritten for the back end.

NEW UNDERNEATH

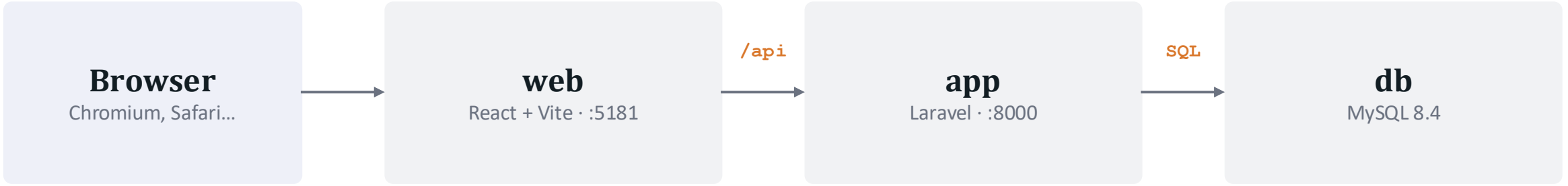
A Laravel API, MySQL, real log in with Sanctum, and three test suites.

RUN IT

cp .env.example .env
docker compose watch
http://localhost:5181

SHOW THE END FIRST

Four containers, one command



Everything the browser sees comes from one address. web forwards /api and /sanctum to app; only app talks to db.

Command	What it runs
`docker compose watch`	db, app and web; edits sync in
`docker compose run --rm unit`	PHPUnit: the rules, no database
`docker compose run --rm api`	Playwright request: every endpoint and error
`docker compose run --rm e2e`	Playwright in Chromium: the pages

The contract first

The midterm built the front end against a written API design. Now time to build the api.

What changed, and what did not

	Midterm (pantry-example-midterm)	Final (pantry-example-final)
Data	web/src/fake-server.js, in memory	Laravel API in api/, rows in MySQL
request() in api.js	calls the fake server	calls fetch()
The pages	Shelf, Item, form	the same files, unchanged
Log in	none (acts as Alex)	Sanctum: log in, register, log out
The rules	web/src/rules.js	Form Requests, a Policy, Stock.php
"Use some"	worked out in the browser, sent with PUT	POST /api/items/{id}/use:

Both sides agree on one shape

GET /api/items/1

```
{
  "data": {
    "id": 1, "user_id": 1, "name": "Milk",
    "amount": 1, "unit": "L", "location": "fridge",
    "best_before": "2026-11-02",
    "status": "due soon", "days_left": 3,
    "owner": { "id": 1, "name": "Alex Rivera" },
    "created_at": "…", "updated_at": "…"
  }
}
```

Green: worked out by the API on every read, never stored.

WHY IT WORKS

The page reads item.status and item.owner.name. It never asks where they came from: fake server or Laravel.

THE RULE

Write the shape down first (the midterm design), then make both sides produce it.

COMMON MISTAKE

Letting Laravel send its default shape (e.g. amount as "1.00") and changing every page to match.

The map from the midterm to api/

#	Do this	Command or file
1	A new Laravel project in api/, next to web/. The pantry final's Dockerfile; db and app services in docker-compose.yml; DB_CONNECTION=mysql	`composer create-project laravel/laravel api` `artisan install:api`
2	A migration for your core resource: every field in API.md, plus user_id → users. A role column on users	`artisan make:model Review -mfc --api`
3	The model with user(); User with reviews(); a factory with one fake value per field	`app/Models/Review.php` `database/factories/ReviewFactory.php`
4	The seeder: three users (one admin) and your midterm's fake data, eight rows or more	`database/seeders/DatabaseSeeder.php` `artisan migrate:fresh --seed`
5	Five routes in one line. GET list and GET one return your API.md JSON; the other three answer 501 until HW8	`Route::apiResource('reviews', …)` `artisan route:list --path=api`
6	This week's PROCESS.md entry. Commit by pull request	`PROCESS.md`

No starter and no autograder this time: we go through each team's api/ together in class the week after. Every command, in order, with a Review example: homework/hw7/SIMPLE.md in the course repo.

Next steps for your team

Step	Do this	Practised in
1	Start api/: a migration, model, factory and seeder for your core resource. GET list and GET one work; the three writes answer 501	HW7, due Fri 11/6 (previous slide)
2	Log in; a policy and Form Requests with your design’s messages; the three write endpoints become real	HW8, assigned Fri 11/6